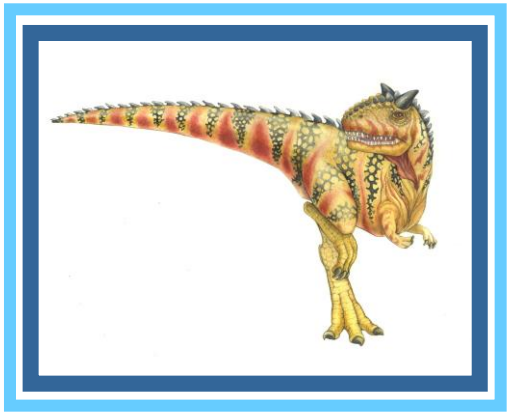
Chapter 7: Synchronization Examples

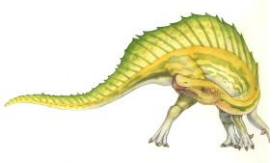


Silberschatz, Galvin and Gagne ©2018



Chapter 7: Synchronization Examples

- Explain the bounded-buffer, readers-writers, and dining-philosophers synchronization problems
- Solve them with semaphores and with monitors
- Judge a solution on four tests: mutual exclusion, no deadlock, no starvation, maximum concurrency
- Describe the elevator algorithm as a monitor-based scheduling problem
- Recognize the POSIX synchronization API (mutex locks, semaphores, condition variables)





Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes

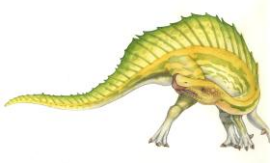
Bounded-Buffer
Producers and consumers share n slots

Readers-Writers
Many readers OR one writer

Dining-Philosophers
Each process needs two shared resources

Elevator Algorithm
Serve requests in sweeps, no starvation

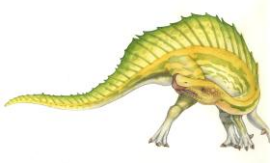
Every solution is judged on: **mutual exclusion**, **no deadlock**, **no starvation**, **maximum concurrency**





Bounded-Buffer Problem

- ***n*** buffers, each can hold one item
- Semaphore **`mutex`** initialized to the value 1
- Semaphore **`full`** initialized to the value 0
- Semaphore **`empty`** initialized to the value n





Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
while (true) {
        ...
        /* produce an item in next_produced */
        ...
    wait(empty);
    wait(mutex);
        ...
        /* add next produced to the buffer */
        ...
    signal(mutex);
    signal(full);
}
```

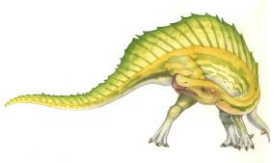




Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
while (true) {
    wait(full);
    wait(mutex);
        ...
        /* remove an item from buffer to next_consumed */
        ...
    signal(mutex);
    signal(empty);
        ...
        /* consume the item in next consumed */
        ...
}
```





Readers-Writers Problem

- A data set is shared among a number of concurrent processes
  - **Readers** – only read the data set; they do ***not*** perform any updates
  - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
  - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities
- Shared Data
  - Data set
  - Semaphore **`rw_mutex`** initialized to 1
  - Semaphore **`mutex`** initialized to 1
  - Integer **`read_count`** initialized to 0





Who May Use the Shared Data?



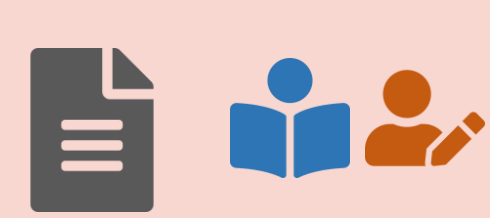
Many readers together

Reading does not change the data



One writer alone

Nobody else may look while it changes



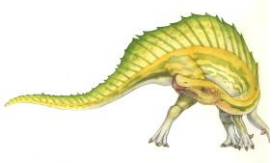
A reader and a writer

The reader could see half-written data



Two writers

Their updates would overwrite each other

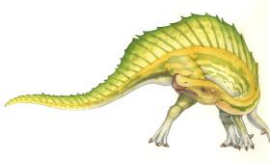




Readers-Writers Problem (Cont.)

- The structure of a writer process

```
while (true) {
      wait(rw_mutex);
         ...
      /* writing is performed */
         ...
   signal(rw_mutex);
}
```





Readers-Writers Problem (Cont.)

- The structure of a reader process

```
while (true){
        wait(mutex);
        read_count++;
        if (read_count == 1)
                wait(rw_mutex);
        signal(mutex);

        ...
        /* reading is performed */
        ...

        wait(mutex);
        read count--;
        if (read_count == 0)
                signal(rw_mutex);
        signal(mutex);

}
```





Readers-Writers Problem (Cont.)

Structure of a Reader Process

```
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
      wait(rw_mutex);
    signal(mutex);
    ENTRY: count up; first reader locks out writers

    ...
    /* reading is performed */
    All readers here at the same time

    wait(mutex);
    read_count--;
    if (read_count == 0)
      signal(rw_mutex);
    signal(mutex);
    EXIT: count down; last reader opens the door
}
```

The two semaphores

- `mutex` guards the counter read_count
- `rw_mutex` guards the data from writers

Only the FIRST reader takes rw_mutex and only the LAST reader gives it back.

First reader R1 arrives: read_count 0 to 1



R2 and R3 skip rw_mutex: read_count = 3



Last reader out: read_count 0, writer enters



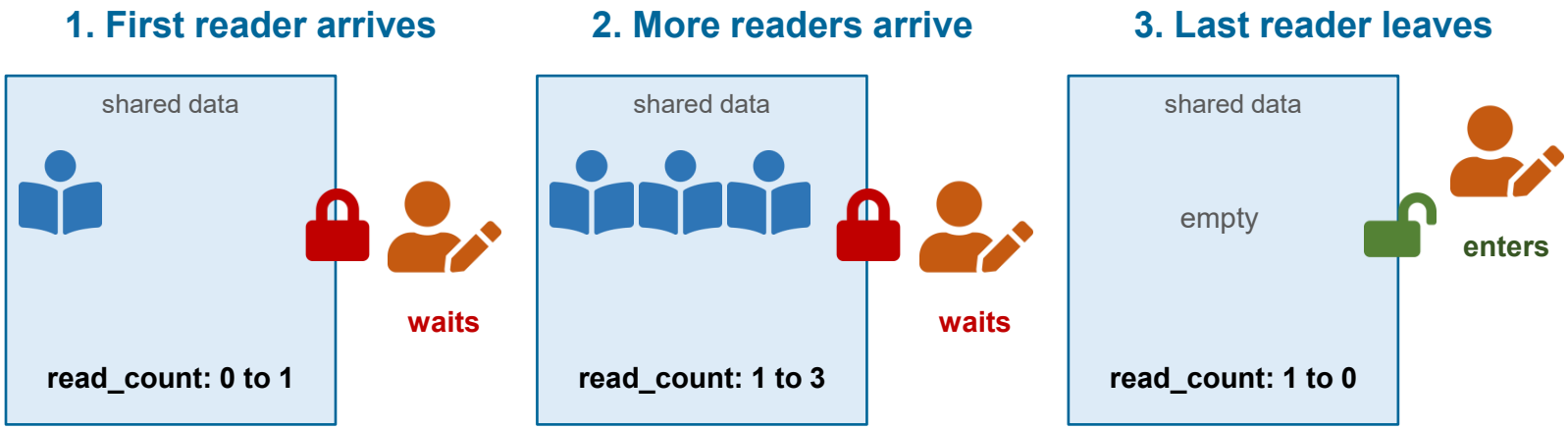
The first reader locks the door against writers, the last reader unlocks it, everyone between reads freely.





The Reader Code as a Room

Think of the shared data as a room. **rw_mutex** is the lock on the door that keeps writers out

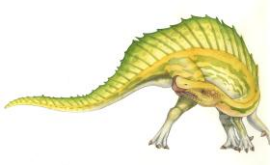


wait(rw_mutex): locks out writers

Already locked: read_count++

signal(rw_mutex): writer enters

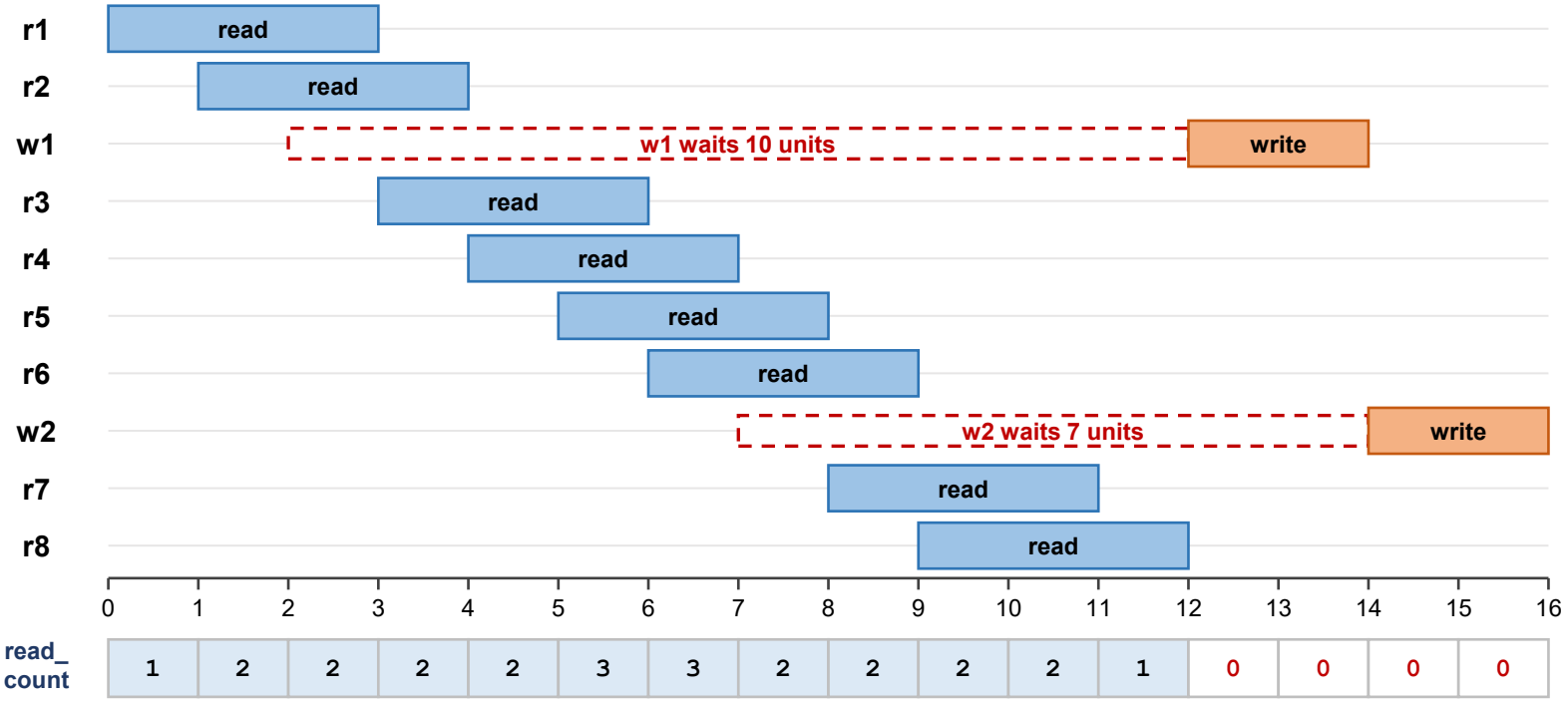
- **mutex** is a second, smaller lock: it protects read_count, so two readers never update the counter at the same time
- Only the first reader and the last reader touch rw_mutex; the readers in between never wait for it



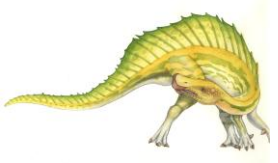


Semaphore Solution: Writers Can Starve

Many readers, two writers. A new reader arrives every 1 or 2 time units, and a read takes 3 units



- read_count stays above 0 from t = 0 to t = 12: a new reader always comes in before the last one leaves
- So nobody calls signal(rw_mutex). If readers never stop coming, w1 and w2 wait forever: **writers starve**



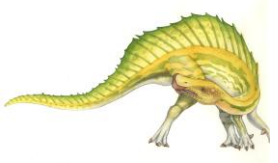


Readers-Writers Problem Variations

- **First variation** (readers preference): no reader waits unless a writer is already writing
  - The semaphore solution on the previous slides is this variation
  - **Writers can starve**: if readers keep arriving, read_count never drops to 0, so signal(rw_mutex) never runs
- **Second variation** (writers preference): once a writer is ready, it writes as soon as possible
  - **Readers can starve**

Goal: maximum concurrency of readers, and **neither** readers nor writers starve

The next slide gives two rules that achieve this





A Second Solution: Stop Writer Starvation

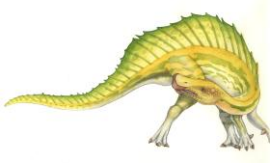
Solution 1: semaphores

- Uses rw_mutex, mutex and read_count
- A reader waits only if a writer is **already writing**
- A **waiting** writer does not stop new readers
- **Writers can starve** while readers keep arriving

Solution 2: monitor

- Uses conditions ok_to_read and ok_to_write
- A new reader also waits if a writer is **waiting**: count(ok_to_write) > 0
- When a writer leaves, **all** waiting readers go in together
- **Nobody starves**: it follows the two rules on the next slide

- Both are correct: mutual exclusion and no deadlock. They differ in **fairness**
- The fair version must know whether a writer is **waiting**. With semaphores that needs extra counters and semaphores and is easy to get wrong; a monitor asks count(ok_to_write) in one line
- So the chapter shows the simple semaphore version first, then the monitor version that passes all four tests





Readers-Writers: Two Rules



Rule 1

A reader may join other readers in the CS **only if no writer is waiting**. When the last reader exits, the waiting writer enters.



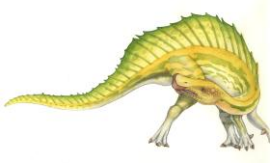
Rule 2

All readers that arrived **while a writer was in the CS** enter before the next writer.

Writers cannot starve

Readers cannot starve

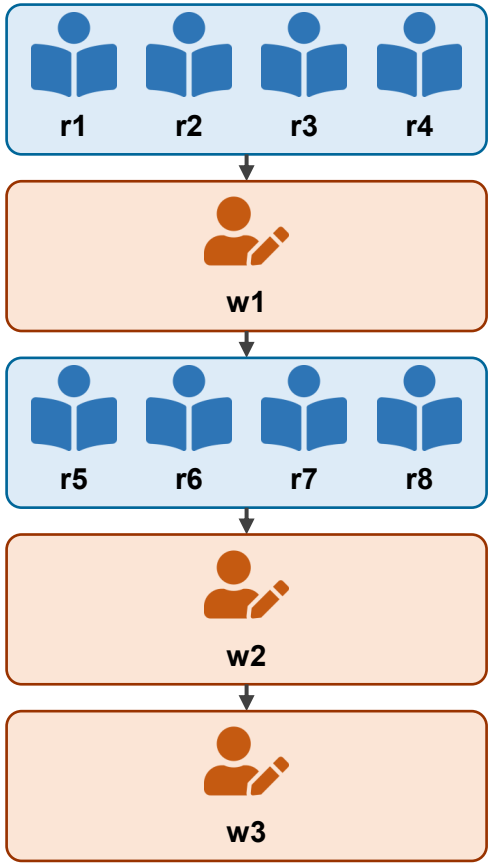
Together the two rules give **maximum concurrency of readers** with **no starvation** of either type





Readers-Writers: Order of Entry

Example: r1, r2, r3, r4 arrive, then w1, r5, r6; while w1 writes, r7, w2, w3, r8 arrive



CS empty, no writer waiting: readers share the CS

w1 arrives: r5, r6 must wait (Rule 1); w1 enters when r4 leaves

w1 exits: every waiting reader enters, even r8 that came after w2, w3 (Rule 2)

Last reader exits: next writer

One writer at a time

Order: (r1 r2 r3 r4), w1, (r5 r6 r7 r8), w2, w3





Monitor Solution to Readers-Writers

```
monitor readers_writers {
   int reading = 0, writing = 0;
   condition ok_to_read, ok_to_write;
```

```
start_read() {
  if (writing > 0 ||
      count(ok_to_write) > 0)
     ok_to_read.wait;
  reading = reading + 1;
  ok_to_read.signal;
}
end_read() {
  reading = reading - 1;
  if (reading == 0)
     ok_to_write.signal;
}
```

```
start_write() {
  if (reading > 0 ||
      writing > 0)
     ok_to_write.wait;
  writing = 1;
}
end_write() {
  writing = 0;
  if (count(ok_to_read) > 0)
     ok_to_read.signal;
  else ok_to_write.signal;
}  }
```

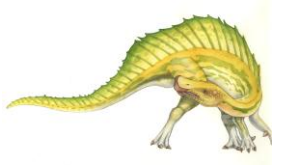
- count(c) = number of processes blocked on condition c
- A waiting writer stops new readers (Rule 1); each woken reader signals the next one, so all waiting readers enter together (Rule 2)





Readers-Writers Monitor Trace

Event	reading	writing	ok_to_read	ok_to_write
r1, r2 call start_read and enter	**2**	**0**		
w1 calls start_write: readers inside, so it waits	**2**	**0**		w1
r3 arrives: a writer is waiting, so it waits (Rule 1)	**2**	**0**	r3	w1
r1, r2 leave; last reader signals w1	**0**	**1**	r3	
r4 and w2 arrive while w1 writes	**0**	**1**	r3 r4	w2
w1 leaves: signals r3, r3 signals r4 (Rule 2)	**2**	**0**		w2
r3, r4 leave; last reader signals w2	**0**	**1**		

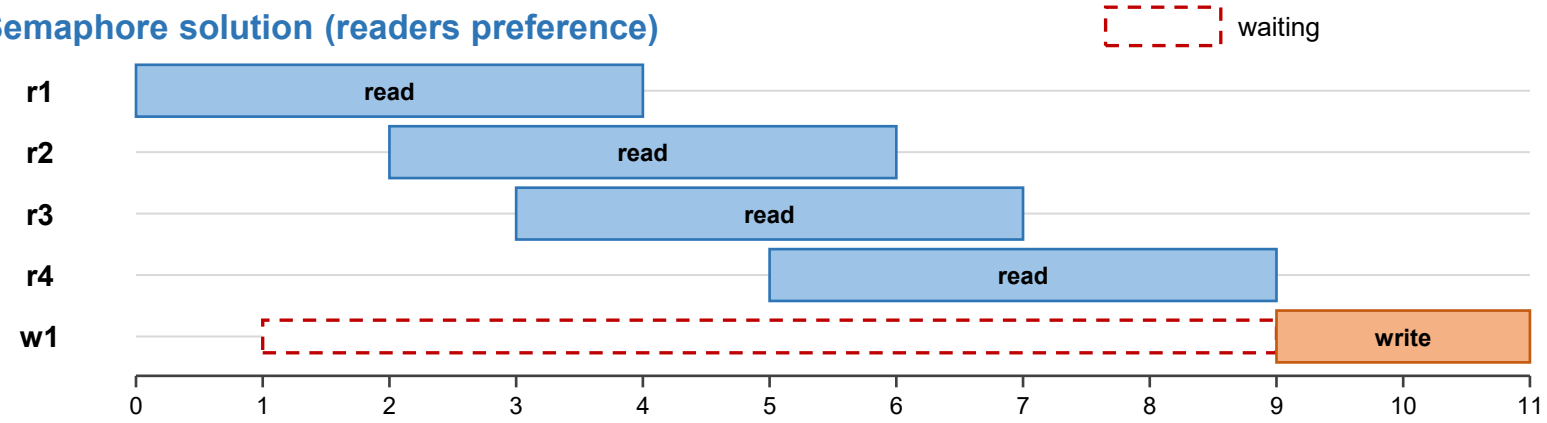




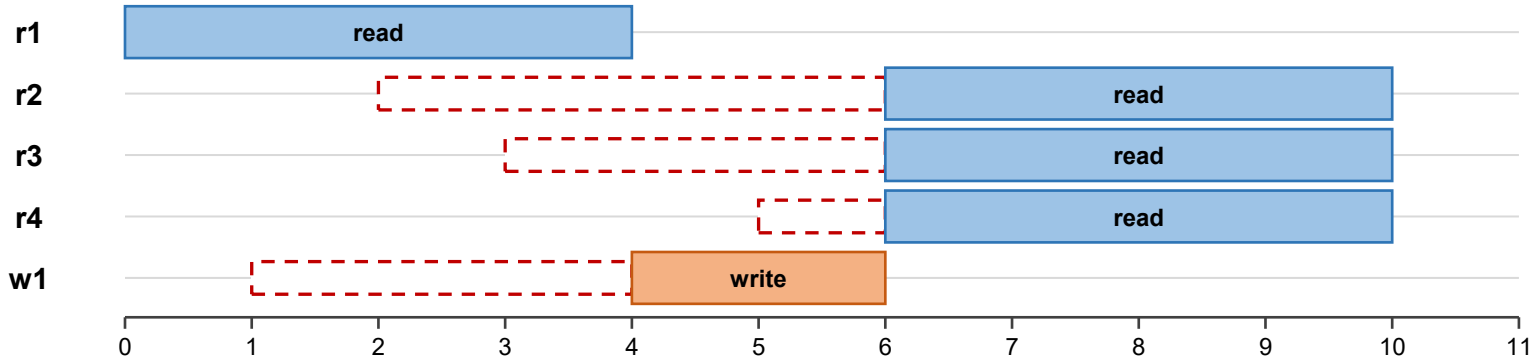
Same Arrivals, Two Solutions

r1 starts reading at t = 0. Then w1 arrives at t = 1, r2 at 2, r3 at 3, r4 at 5. A read takes 4 time units, a write takes 2

Semaphore solution (readers preference)



Monitor solution (two rules)



- Semaphores: w1 waits 8 units, and longer with every new reader. Monitor: w1 waits 3, then r2, r3, r4 read **together** (Rule 2)





Practice: Order of Readers and Writers

- Monitor solution. Writer w1 is writing. These requests arrive in order:

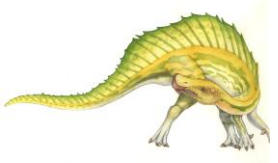
  w2, r1, r2, w3, r3, w4

- (a) In which order are they processed? Which readers read together?
- (b) r4 and r5 arrive while w2 is writing. When do they run?

Answers

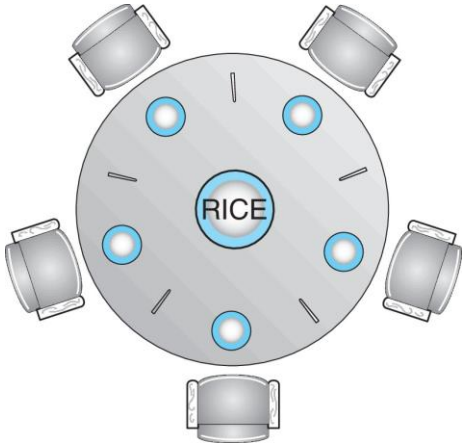
(a) (r1, r2, r3), w2, w3, w4 – all waiting readers go first, together (Rule 2)

(b) Between w2 and w3: they arrived while a writer was in the CS





Dining-Philosophers Problem



- Philosophers spend their lives alternating thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers
  - Shared data
    - Bowl of rice (data set)
    - Semaphore chopstick [5] initialized to 1





Dining-Philosophers Problem Algorithm

- Semaphore Solution
- The structure of Philosopher *i*:

```
while (true){
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

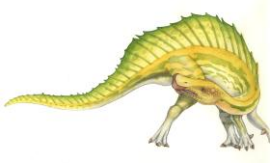
     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );

     /* think for awhile */


}
```

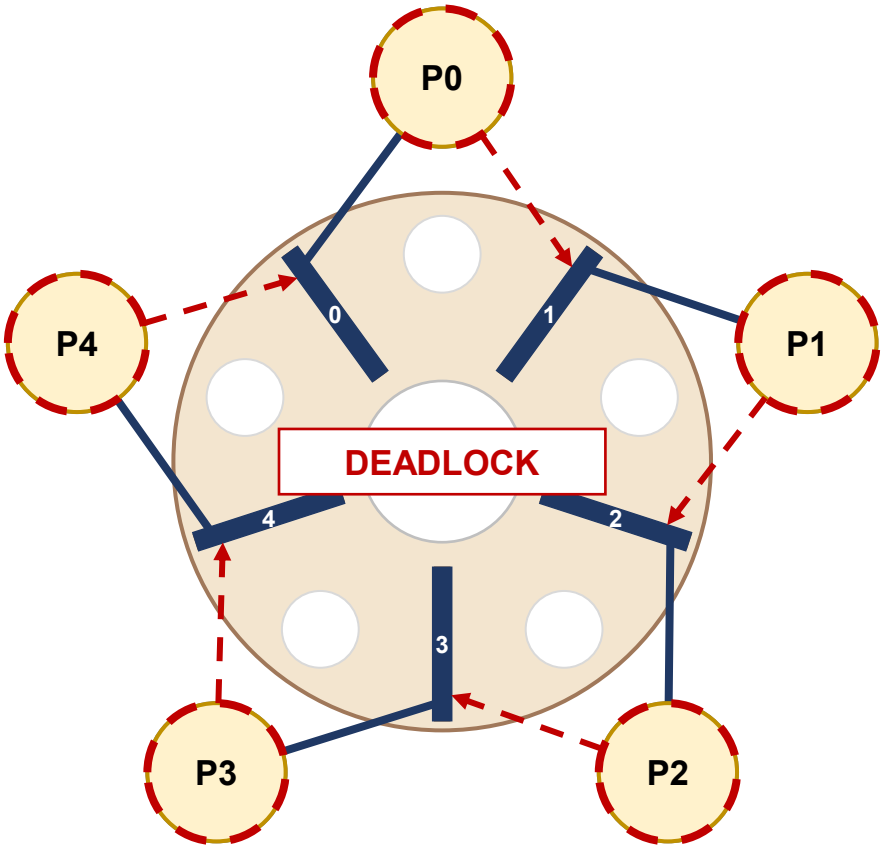
- What is the problem with this algorithm?





Why Dining Philosophers Can Deadlock

5 philosophers, 5 chopsticks. To eat, Pi needs chopstick[i] **and** chopstick[(i+1) % 5]



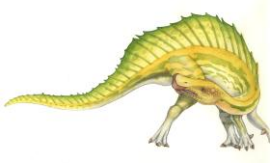
1. All get hungry at once. Each runs wait(chopstick[i]) and picks up its **left** chopstick

2. Each runs wait(chopstick[(i+1) % 5]). That chopstick is in the **neighbor's** hand, so everyone **blocks**

3. Nobody eats, so nobody calls signal(). P0 waits for P1, P1 for P2, ... P4 for P0: a **circle** of waiting, **deadlock**

5 hands hold 1 chopstick each: **0 left over**. Each philosopher needs 2, so nobody can finish

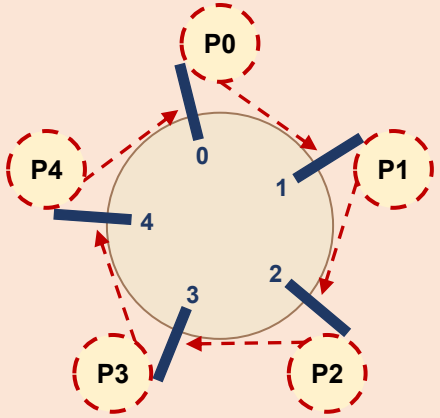
Slide show: click 7 times to play the steps





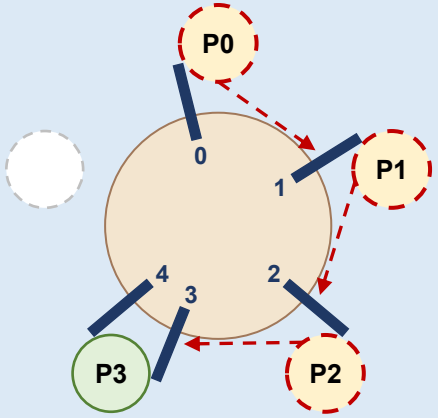
Why Only 5 Chopsticks Is the Problem

Everyone holds one



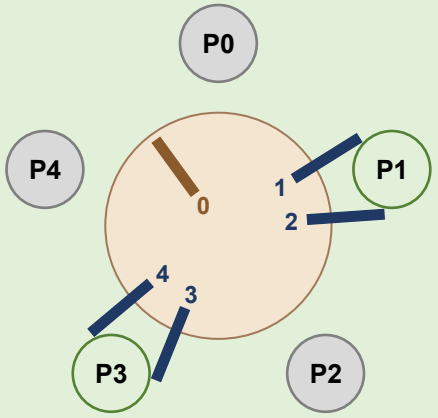
- 5 held, **0 free**
- Nobody can get a 2nd one
- **Deadlock**

Only 4 may sit



- 4 hold one each, **1 free**
- P3 takes the free one and eats, then releases
- No deadlock

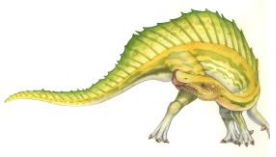
Best case



- P1 and P3 eat: 4 chopsticks
- 1 left over, but nobody can pair it
- **At most 2 eat at once**

- There are enough chopsticks for 2 to eat. The trouble is **holding one while waiting for the other**
- Every solution on the next slides breaks this waiting circle in a different way

Numbers = chopstick[i]. Dark = held by the philosopher beside it. Dashed red arrow = waiting for it





Two Deadlock-Free Approaches

Goal: **no deadlock**, and any two non-adjacent philosophers can always eat at the same time. Forks are semaphores f[0..4] = 1

Approach 1: both forks in a CS

```
while (1) {
  think
  P(mutex)
    P(f[i])
    P(f[i+1 mod 5])
  V(mutex)
  eat
  V(f[i]); V(f[i+1 mod 5])
}
```

- No deadlock: a philosopher gets both forks before anyone else tries
- **Concurrency violated**: one philosopher blocked on a fork while holding mutex blocks everyone else

Approach 2: lower-numbered fork first

```
while (1) {
  think
  P(f[min(i, i+1 mod 5)])
  P(f[max(i, i+1 mod 5)])
  eat
  V(f[i]); V(f[i+1 mod 5])
}
```

- p[4] takes f[0] before f[4], so the circular wait cannot close: no deadlock
- **Concurrency can still be violated**: philosophers can block in a chain while only one eats



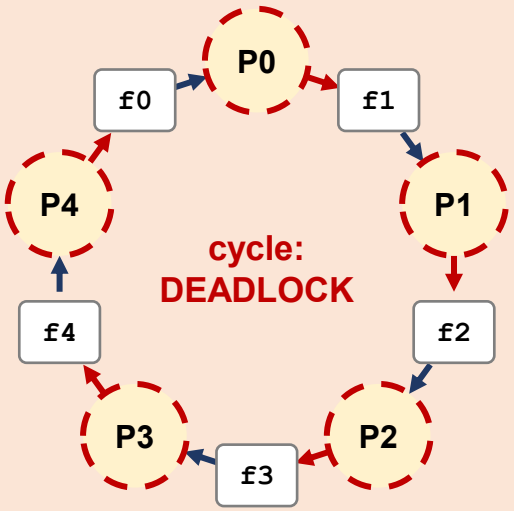


Resource Ordering Breaks the Circular Wait

All five philosophers get hungry at the same moment, and each one reaches for its **first** fork

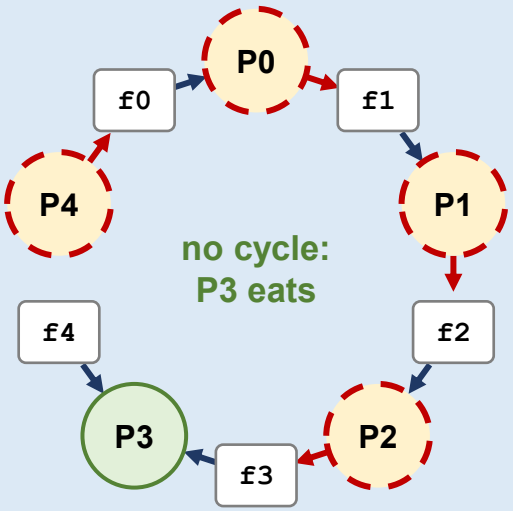
fork → philosopher: holds it philosopher → fork: waiting for it

No ordering: f[i], then f[i+1 mod 5]



Every Pi holds f[i] and waits for f[i+1]. The waits go all the way around: nobody can ever continue

Ordering: lower-numbered fork first



P4 asks for **f0 first**: min(4, 0) = 0. f0 is taken, so P4 waits **holding nothing**, f4 stays free and P3 eats

Pick order	P0	P1	P2	P3	P4
1st P()	f0	f1	f2	f3	f0
2nd P()	f1	f2	f3	f4	f4

`P(f[min(i, i+1 mod 5)])` then
`P(f[max(...)])`: only P4 is reversed

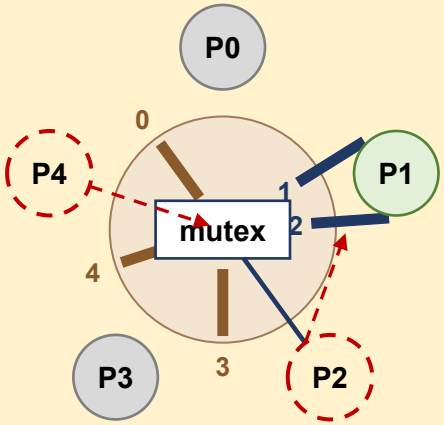
Waits only go to a higher-numbered fork, so they never loop back: no circular wait, no deadlock





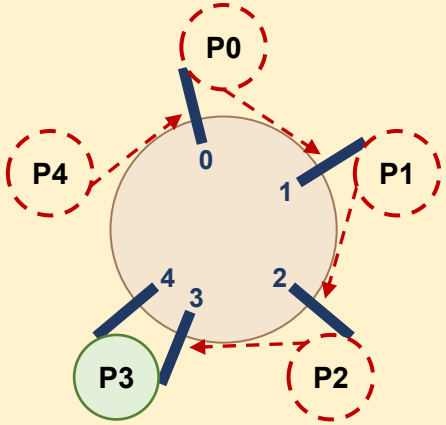
What Goes Wrong with Approach 1 and 2

Approach 1: both in a critical section



- P1 eats with chopsticks 1 and 2
- P2 locked the mutex, then waits for chopstick 2
- P4 is hungry and chopsticks 4 and 0 are free, but it waits for the mutex
- **Only 1 eats**, though P1 and P4 could eat together

Approach 2: lower number first



- P3 eats with chopsticks 3 and 4
- P0, P1, P2 each hold their lower chopstick and wait for the next one
- P4 waits for chopstick 0 and holds nothing
- **Only P3 eats**: the rest wait in a line

- Both are deadlock-free, but fewer philosophers eat at once than could. The monitor solution fixes this

Numbers = chopstick[i]. Dark = held by the philosopher beside it. Dashed red arrow = waiting for it





Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
enum { THINKING, HUNGRY, EATING } state [5] ;
condition self [5];

void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self[i].wait;
}

    void putdown (int i) {
        state[i] = THINKING;
                    // test left and right neighbors
         test((i + 4) % 5);
         test((i + 1) % 5);
}
```

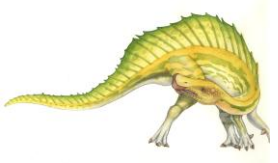




Solution to Dining Philosophers (Cont.)

```
void test (int i) {
        if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
             state[i] = EATING ;
       self[i].signal () ;
        }
   }

       initialization_code() {
       for (int i = 0; i < 5; i++)
       state[i] = THINKING;
     }
}
```





Solution to Dining Philosophers (Cont.)

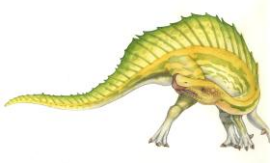
- Each philosopher *i* invokes the operations **`pickup()`** and **`putdown()`** in the following sequence:

```
DiningPhilosophers.pickup(i);

    /** EAT **/

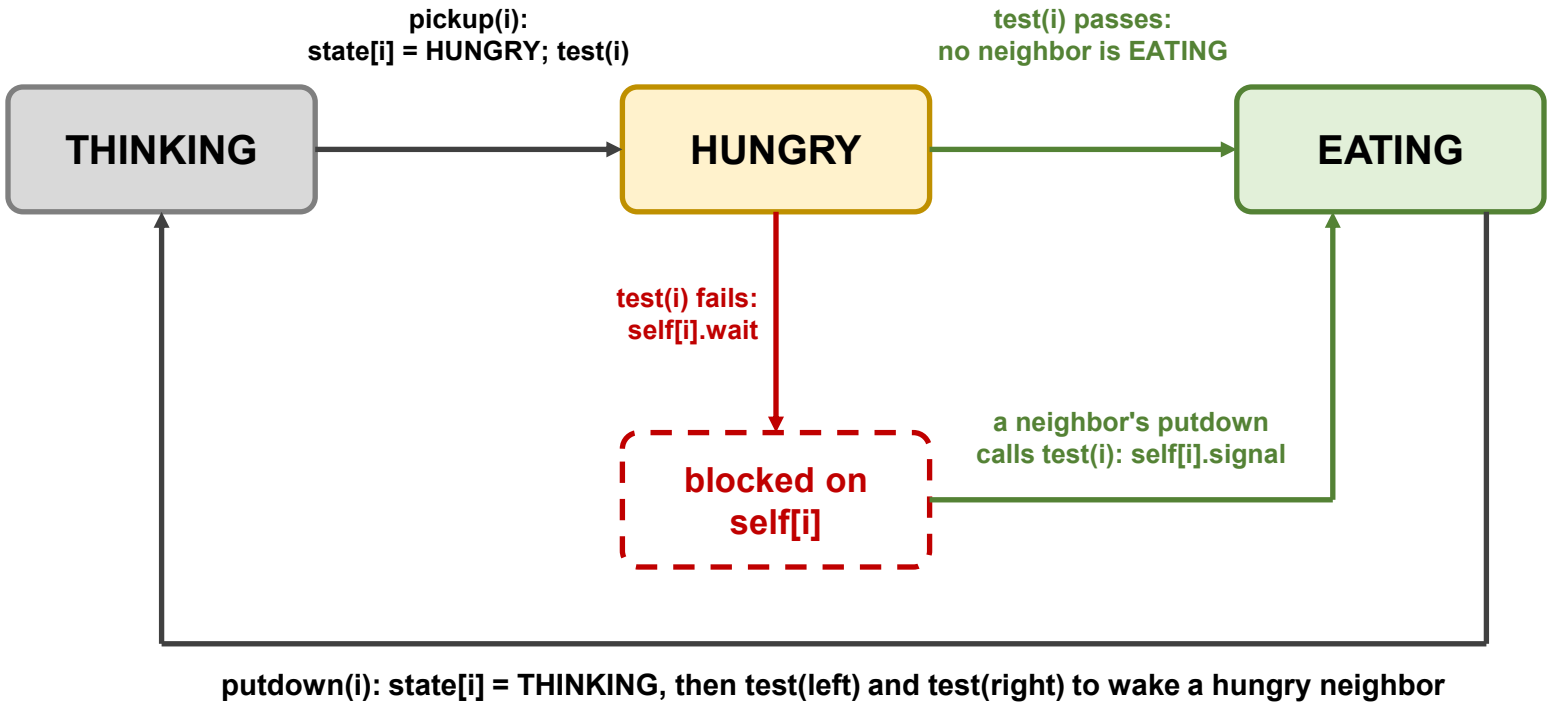
DiningPhilosophers.putdown(i);
```

- No deadlock, but starvation is possible





Inside the Dining-Philosophers Monitor

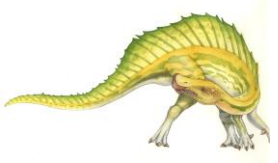


putdown(i): state[i] = THINKING, then test(left) and test(right) to wake a hungry neighbor

What test(i) checks



Only one process runs inside the monitor, so a philosopher checks both neighbors and takes both forks in one step: no deadlock





Dining-Philosophers Monitor Trace

p0	p1	p2	p3	p4	What happens (T thinking, H hungry, E eating)
T	T	T	T	T	Start: everyone is THINKING
T	E	T	T	T	p1 calls pickup(1): HUNGRY, test(1): p0 and p2 not eating, so p1 is **EATING**
T	E	T	E	T	p3 calls pickup(3): test(3) passes, **EATING**. p1 and p3 are not neighbors, so both eat
T	E	H	E	T	p2 calls pickup(2): HUNGRY, test(2) fails (p1, p3 eating), so **self[2].wait**
T	T	H	E	T	p1 calls putdown(1): THINKING. test(0): p0 not hungry. test(2): p3 still eats, p2 keeps waiting
E	T	H	E	T	p0 calls pickup(0): p4 and p1 not eating, so p0 is **EATING** (p2 is waiting, not eating)
E	T	E	T	T	p3 calls putdown(3): THINKING. test(2): p1, p3 not eating, so p2 is **EATING** and self[2].signal wakes it

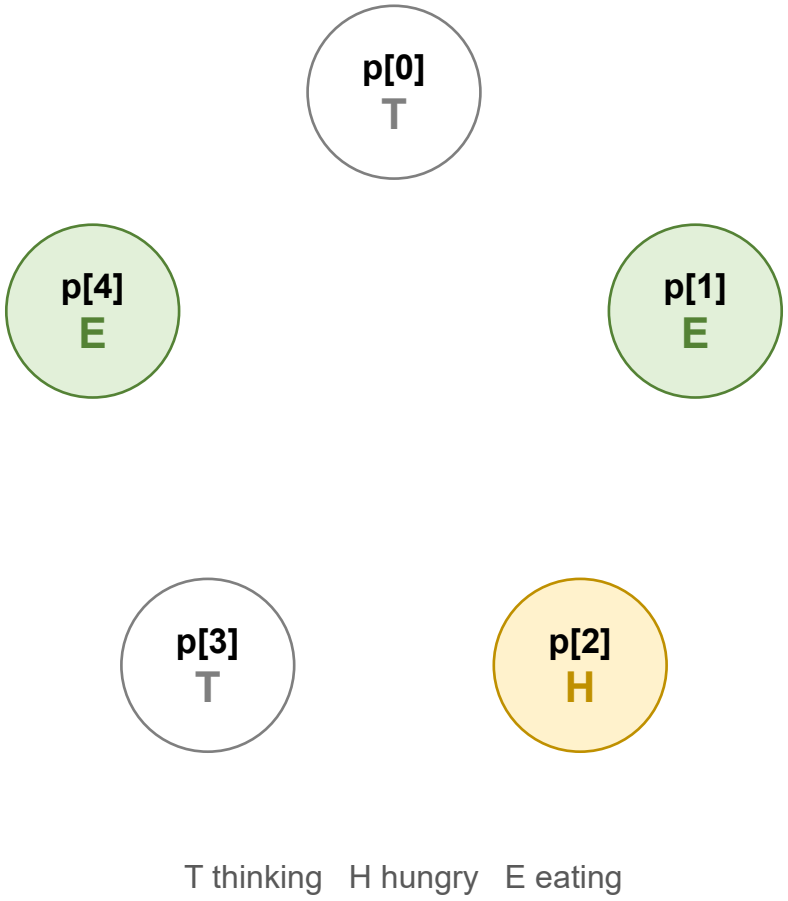
made the call blocked on self[i]

End: p0 and p2 eat together

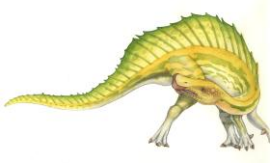




Why the Monitor Solution Works

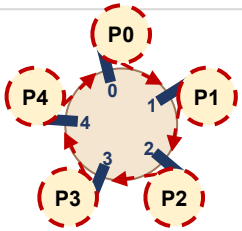
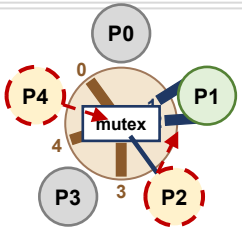
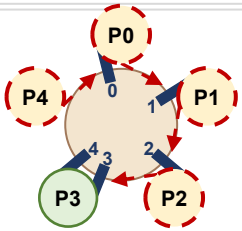
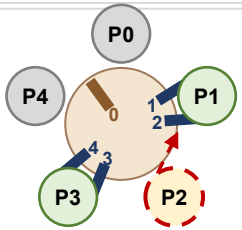


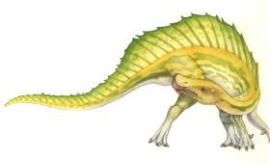
- Each p[i] is **T**, **H** or **E**; forks are not stored at all
- pickup(i): set H, test(i); wait on self[i] if not E
- test(i): H becomes E only if **neither neighbor is eating**
- putdown(i): set T, then test both neighbors to wake them
- **No deadlock**: both forks are taken in one step (H to E)
- **Starvation possible**: p[1] and p[3] can take turns eating forever while p[2] waits





Dining-Philosophers Solutions: Summary

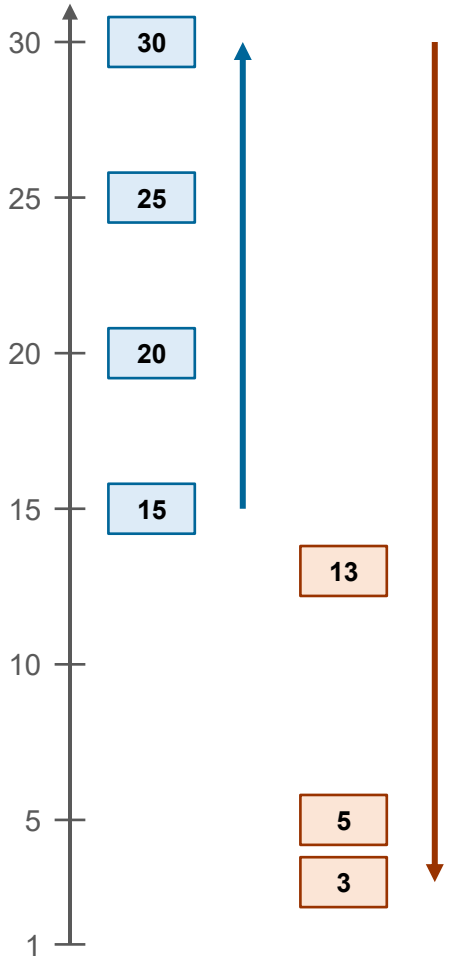
Picture	Solution (where in the slides)	What goes wrong
	**1. Naive semaphores**: wait(left), then wait(right) "Dining-Philosophers Problem Algorithm"	**Deadlock**: all pick up the left one and wait forever
	**2. Both chopsticks inside a critical section** (one mutex) Approach 1 in "Two Deadlock-Free Approaches"	No deadlock, but **only 1 eats**: P2 waits inside the CS, so P4 is stuck even though its chopsticks are free
	**3. Lower-numbered chopstick first** (only P4 is reversed) Approach 2, "Resource Ordering Breaks the Circular Wait"	No deadlock. But they can wait in a line: P4 waits for P0, P0 for P1, P1 for P2, P2 for P3. **Only P3 eats**
	**4. Monitor: take both or none** (test() checks both neighbors) "Monitor Solution to Dining Philosophers" to "Why the Monitor Solution Works"	No deadlock and 2 can eat, but **starvation possible**: P1 and P3 can take turns while P2 waits





The Elevator Algorithm

- A disk has n tracks; read/write requests arrive in any order
- Goal: **minimize head travel** while **preventing starvation**
- The head acts like an elevator in an n-floor building:
  - Keep moving in the current direction while requests remain that way
  - No more requests ahead: reverse and serve the rest in the other order
- Same idea as SCAN disk scheduling







Elevator Algorithm as a Monitor

```
monitor elevator {  int busy = 0, position, direction = up;
                    condition upsweep, downsweep;
```

```
move_to(dest) {
  if (busy)
    if (position < dest ||
        (position == dest &&
         direction == up))
      upsweep.wait(dest);
    else
      downsweep.wait(-dest);
  busy = 1;
  position = dest;
}
```

```
release() {
  busy = 0;
  if (direction == up)
    if (!empty(upsweep))
      upsweep.signal;
    else { direction = down;
           downsweep.signal; }
  else if (!empty(downsweep))
      downsweep.signal;
    else { direction = up;
           upsweep.signal; }
}  }
```

- **Priority wait** c.wait(p): the process with the smallest p is resumed first
- upsweep sorted ascending by dest; downsweep uses -dest, so it is sorted descending
- A request for the current floor joins the queue of the current direction, so it is served in this sweep

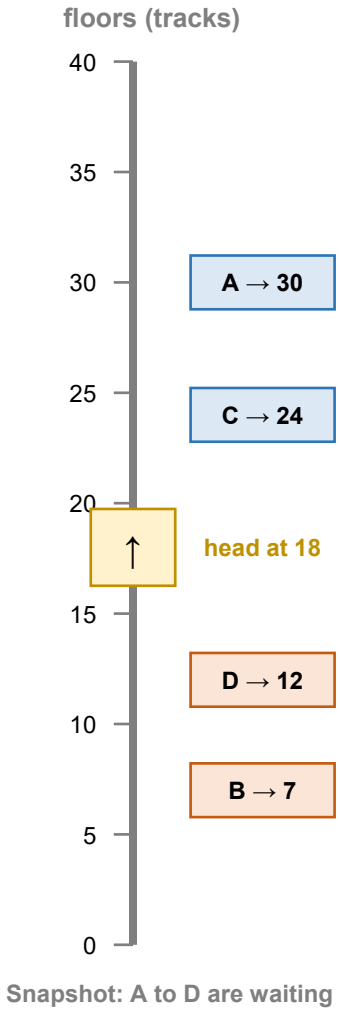
- Head free: take it (busy = 1, position = dest)
- Busy, dest above (or same floor, going up): **upsweep**.wait(dest)
- Busy, otherwise: **downsweep**.wait(-dest)

- busy = 0
- Queue for the current direction not empty: wake its first process
- Else reverse direction and wake the first of the other queue





Inside the Elevator Monitor




```
monitor elevator
```

busy = 1 | position = 18 | direction = up

upsweep ← 24 C | 30 A — smallest floor first

downsweep ← 12 D | 7 B — largest floor first

next out

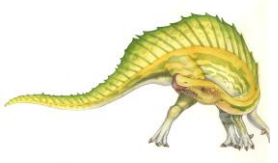
Priority waits: c.wait(p) wakes the smallest p. downsweep stores -dest

move_to(dest)

- Head free: take it (busy = 1, position = dest)
- Busy, dest above (or same floor, going up): **upsweep**.wait(dest)
- Busy, otherwise: **downsweep**.wait(-dest)

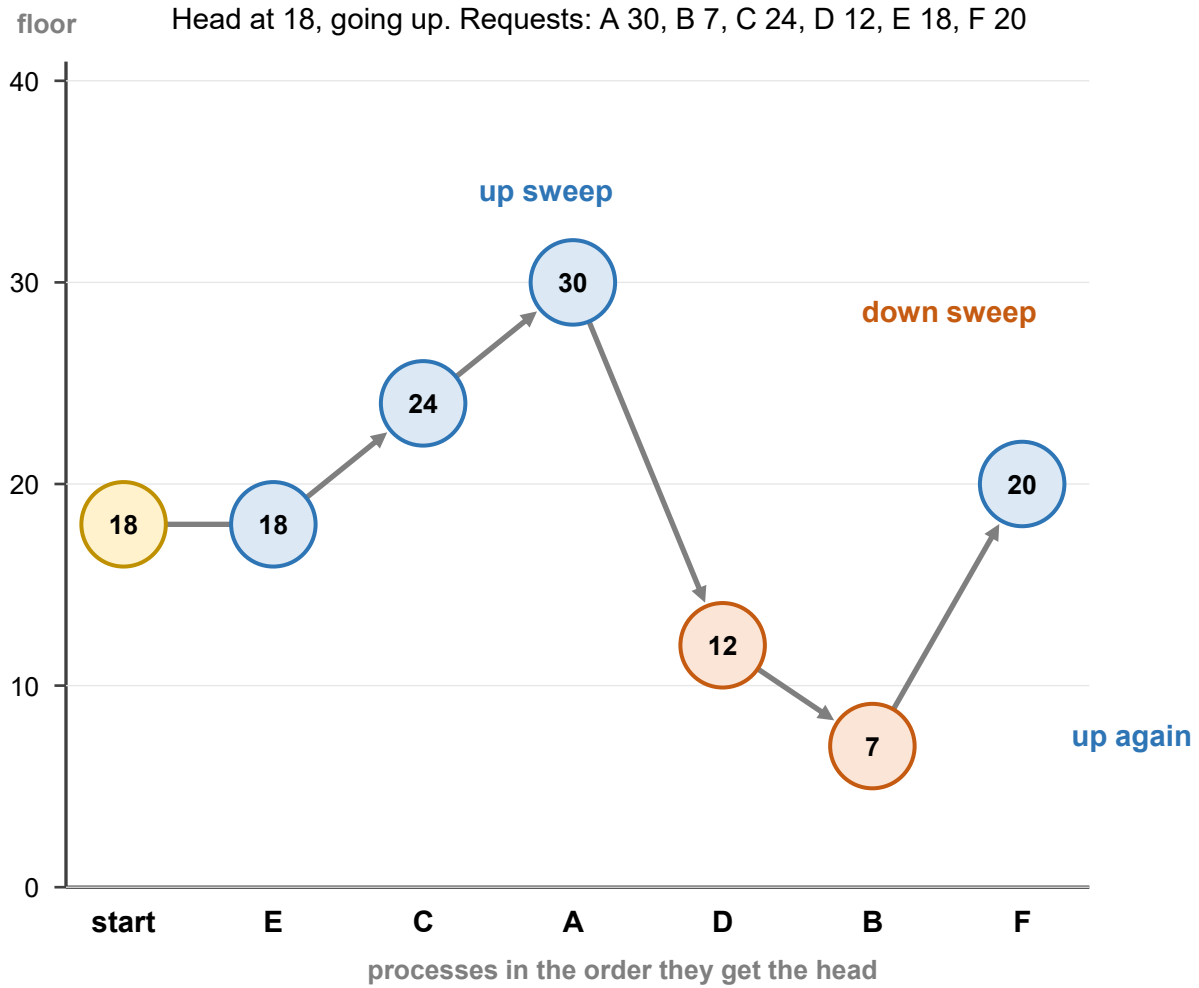
release()

- busy = 0
- Queue for the current direction not empty: wake its first process
- Else reverse direction and wake the first of the other queue





Elevator Example: Path of the Head



- Head travel with the monitor:
  - 0 + 6 + 6 + 18 + 5 + 13 = **48** floors
- In arrival order (A, B, C, D, E, F):
  - 12 + 23 + 17 + 12 + 6 + 2 = **72** floors
- No starvation: a request waits at most one sweep in each direction





Elevator Example and Practice

Head busy at floor 15, moving **up**. Requests arrive for 25, 30, 20, 5, 13, 3, 15

upsweep	**15**	**20**	**25**	**30**
downsweep	**13**	**5**	**3**	

Service order:

15, 20, 25, 30, 13, 5, 3

- Practice: the head is at floor 20, moving **down**. Which queue gets each request?
  - move_to(1) move_to(22) move_to(20)

Answers

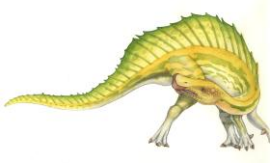
move_to(1): downsweep (below, same direction)
move_to(22): upsweep (above, waits for the next up sweep)
move_to(20): downsweep (same floor, current direction is down)





POSIX Synchronization

- POSIX API provides
  - mutex locks
  - semaphores
  - condition variable
- Widely used on UNIX, Linux, and macOS





POSIX Mutex Locks

- Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

- Acquiring and releasing the lock

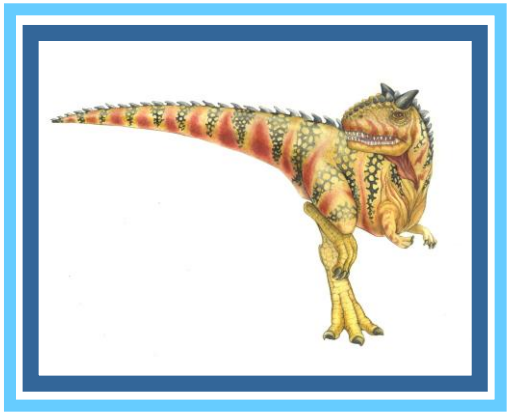
```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```



End of Chapter 7



Silberschatz, Galvin and Gagne ©2018